GALAHAD RQS

USER DOCUMENTATION GALAHAD Optimization Library version 5.5

1 SUMMARY

Given real n by n symmetric matrices $\mathbf{H}$ and $\mathbf{M}$ (with $\mathbf{M}$ diagonally dominant), , another real m by n matrix $\mathbf{A}$, a real n vector $\mathbf{c}$ and scalars $\sigma > 0$, $p > 2$ and f, this package finds a **global minimizer of the norm-regularised quadratic objective function $\frac{1}{2}\mathbf{x}^T\mathbf{H}\mathbf{x} + \mathbf{c}^T\mathbf{x} + f + \frac{1}{p}\sigma\|\mathbf{x}\|_{\mathbf{M}}^p$, where the vector $\mathbf{x}$ may additionally be required to satisfy $\mathbf{A}\mathbf{x} = \mathbf{0}$,** and where the $\mathbf{M}$-norm of $\mathbf{x}$ is $\|\mathbf{x}\|_{\mathbf{M}} = \sqrt{\mathbf{x}^T\mathbf{M}\mathbf{x}}$. This problem commonly occurs as a subproblem in nonlinear optimization calculations. The matrix $\mathbf{M}$ need not be provided in the commonly-occurring ℓ_2-regularisation case for which $\mathbf{M} = \mathbf{I}$, the n by n identity matrix.

Factorization of matrices of the form $\mathbf{H} + \lambda\mathbf{M}$—or

$$\begin{pmatrix} \mathbf{H} + \lambda\mathbf{M} & \mathbf{A}^T \\ \mathbf{A} & \mathbf{0} \end{pmatrix} \tag{1.1}$$

in cases where $\mathbf{A}\mathbf{x} = \mathbf{0}$ is imposed—for a succession of scalars λ will be required, so this package is most suited for the case where such a factorization may be found efficiently. If this is not the case, the package `GALAHAD_GLRT` may be preferred.

ATTRIBUTES — Versions: `GALAHAD_RQS_single`, `GALAHAD_RQS_double`. **Uses:** `GALAHAD_CLOCK`, `GALAHAD_SYMBOLS`, `GALAHAD_SPACE`, `GALAHAD_RAND`, `GALAHAD_NORMS`, `GALAHAD_ROOTS`, `GALAHAD_SPECFILE`, `GALAHAD_SLS`, `GALAHAD_IR`, `GALAHAD_MOP` **Date:** November 2008. **Origin:** N. I. M. Gould, H. S. Thorne, Rutherford Appleton Laboratory, and D. P. Robinson, Oxford University. **Language:** Fortran 95 + TR 15581 or Fortran 2003. **Parallelism:** Some options may use OpenMP and its runtime library.

2 HOW TO USE THE PACKAGE

The package is available with single, double and (if available) quadruple precision reals, and either 32-bit or 64-bit integers. Access to the 32-bit integer, single precision version requires the `USE` statement

```
USE GALAHAD_RQS_single
```

with the obvious substitution `GALAHAD_RQS_double`, `GALAHAD_RQS_quadruple`, `GALAHAD_RQS_single_64`, `GALAHAD_RQS_double_64` and `GALAHAD_RQS_quadruple_64` for the other variants.

If it is required to use more than one of the modules at the same time, the derived types `SMT_TYPE`, `RQS_control_type`, `RQS_history_type`, `RQS_inform_type`, `RQS_data_type`, (Section 2.4) and the subroutines `RQS_initialize`, `RQS_solve`, `RQS_terminate` (Section 2.5) and `RQS_read_specfile` (Section 2.7) must be renamed on one of the `USE` statements.

2.1 Matrix storage formats

The matrices $\mathbf{H}$ and (if required) $\mathbf{M}$ and $\mathbf{A}$ may be stored in a variety of input formats.

2.1.1 Dense storage format

The matrix $\mathbf{H}$ is stored as a compact dense matrix by rows, that is, the values of the entries of each row in turn are stored in order within an appropriate real one-dimensional array. Since $\mathbf{H}$ is symmetric, only the lower triangular part (that is the part h_{ij} for $1 \leq j \leq i \leq n$) need be held. In this case the lower triangle should be stored by rows, that is

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

component $i*(i-1)/2+j$ of the storage array `H%val` will hold the value h_{ij} (and, by symmetry, h_{ji}) for $1 \leq j \leq i \leq n$. The same is true for **M** if it is used. If **A** is used, the entire matrix must be supplied, and component $n*(i-1)+j$ of the storage array `A%val` will hold the value a_{ij} for $i = 1,\ldots,m$, $j = 1,\ldots,n$.

2.1.2 Sparse co-ordinate storage format

Only the nonzero entries of the matrices are stored. For the l-th entry of **H**, $1 \leq j \leq i \leq n$, its row index i, column index j and value h_{ij} are stored in the l-th components of the integer arrays `H%row`, `H%col` and real array `H%val`, respectively. Note that only the entries in the lower triangle should be stored. The same scheme may be used for **M** if it is required. If **A** is used, the entire matrix must be supplied using the same scheme in the integer arrays `A%row`, `A%col` and real array `A%val`.

2.1.3 Sparse row-wise storage format

Again only the nonzero entries are stored, but this time they are ordered so that those in row i appear directly before those in row $i+1$. For the i-th row of **H**, the i-th component of the integer array `H%ptr` holds the position of the first entry in this row, while `H%ptr` $(m+1)$ holds the total number of entries plus one. The column indices j, $1 \leq j \leq i$, and values h_{ij} of the entries in the i-th row are stored in components $l =$ `H%ptr`(i), ...,`H%ptr` $(i+1)-1$ of the integer array `H%col`, and real array `H%val`, respectively. Note that as before only the entries in the lower triangle should be stored. For sparse matrices, this scheme almost always requires less storage than its predecessor. This scheme may also be used for **M** and **A** if they are required, excepting that for **A** the whole matrix must be stored.

2.1.4 Diagonal storage format

If **H** is diagonal (i.e., $h_{ij} = 0$ for all $1 \leq i \neq j \leq n$) only the diagonals entries h_{ii}, $1 \leq i \leq n$, need be stored, and the first n components of the array `H%val` may be used for the purpose. The same applies to **M** if it is required. This scheme is inappropriate and thus unavailable for **A**.

2.1.5 Scaled-identity-matrix storage format

If **H** is a scalar multiple of the identity matrix (i.e., $h_{ii} = h_{11}$ and $h_{ij} = 0$ for all $1 \leq i \neq j \leq n$) only the first diagonal entry h_{11} needs be stored, and the first component of the array `H%val` may be used for the purpose. The same applies to **M** if it is required, but again, there is no sensible equivalent for the non-square **A**.

2.1.6 Identity-matrix storage format

If **H** is the identity matrix (i.e., $h_{ii} = 1$ and $h_{ij} = 0$ for all $1 \leq i \neq j \leq n$), no explicit entries needs be stored. The same is true for **M** but not for the non-square **A**.

2.1.7 Zero-matrix storage format

If $\mathbf{H} = \mathbf{0}$ (i.e., $h_{ij} = 0$ for all $1 \leq i, j \leq n$), no explicit entries needs be stored. The same applies both to **M** and **A**, although there would seem little real justification for using $\mathbf{A} = 0$.

2.2 Real and integer kinds

We use the terms integer and real to refer to the fortran keywords `REAL(rp_)` and `INTEGER(ip_)`, where `rp_` and `ip_` are the relevant kind values for the real and integer types employed by the particular module in use. The former are equivalent to default `REAL` for the single precision versions, `DOUBLE PRECISION` for the double precision cases and quadruple-precision if 128-bit reals are available, and correspond to `rp_ = real32`, `rp_ = real64` and `rp_ =`

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

real128 respectively as defined by the fortran iso_fortran_env module. The latter are default (32-bit) and long (64-bit) integers, and correspond to ip_ = int32 and ip_ = int64, respectively, again from the iso_fortran_env module.

2.3 Parallel usage

OpenMP may be used by the GALAHAD_RQS package to provide parallelism for some solvers in shared memory environments. See the documentation for the GALAHAD package SLS for more details. To run in parallel, OpenMP must be enabled at compilation time by using the correct compiler flag (usually some variant of -openmp). The number of threads may be controlled at runtime by setting the environment variable OMP_NUM_THREADS.

MPI may also be used by the package to provide parallelism for some solvers in a distributed memory environment. To use this form of parallelism, MPI must be enabled at runtime by using the correct compiler flag (usually some variant of -lmpi). Although the MPI process will be started automatically when required, it should be stopped by the calling program once no further use of this form of parallelism is needed. Typically, this will be via statements of the form

```
CALL MPI_INITIALIZED( flag, ierr )
IF ( flag ) CALL MPI_FINALIZE( ierr )
```

The code may be compiled and run in serial mode.

2.4 The derived data types

Six derived data types are accessible from the package.

2.4.1 The derived data type for holding matrices

The derived data type SMT_TYPE is used to hold the matrices **H** and perhaps **M** and/or **A**. The components of SMT_TYPE used here are:

m is a scalar component of type INTEGER(ip_), that holds the number of rows in the matrix.

n is a scalar component of type INTEGER(ip_), that holds the number of columns in the matrix.

ne is a scalar variable of type INTEGER(ip_), that holds the number of matrix entries.

type is a rank-one allocatable array of type default CHARACTER, that is used to indicate the matrix storage scheme used. Its precise length and content depends on the type of matrix to be stored.

val is a rank-one allocatable array of type REAL(rp_) and dimension at least ne, that holds the values of the entries. Each pair of off-diagonal entries $h_{ij} = h_{ji}$ of the *symmetric* matrix **H** is represented as a single entry (see §2.1.1–2.1.3). Any duplicated entries that appear in the sparse co-ordinate or row-wise schemes will be summed.

row is a rank-one allocatable array of type INTEGER(ip_), and dimension at least ne, that may hold the row indices of the entries. (see §2.1.2).

col is a rank-one allocatable array of type INTEGER(ip_), and dimension at least ne, that may hold the column indices of the entries (see §2.1.2–2.1.3).

ptr is a rank-one allocatable array of type INTEGER(ip_), and dimension at least n + 1, that may holds the pointers to the first entry in each row (see §2.1.3).

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

2.4.2 The derived data type for holding control parameters

The derived data type `RQS_control_type` is used to hold controlling data. Default values may be obtained by calling `RQS_initialize` (see Section 2.5.1). The components of `RQS_control_type` are:

`error` is a scalar variable of type `INTEGER(ip_)`, that holds the stream number for error messages. Printing of error messages in `RQS_solve` and `RQS_terminate` is suppressed if `error` ≤ 0. The default is `error = 6`.

`out` is a scalar variable of type `INTEGER(ip_)`, that holds the stream number for informational messages. Printing of informational messages in `RQS_solve` is suppressed if `out` < 0. The default is `out = 6`.

`print_level` is a scalar variable of type `INTEGER(ip_)`, that is used to control the amount of informational output which is required. No informational output will occur if `print_level` ≤ 0. If `print_level = 1` a single line of output will be produced for each iteration of the process. If `print_level` ≥ 2 this output will be increased to provide significant detail of each iteration. The default is `print_level = 0`.

`dense_factorization` is a scalar variable of type `INTEGER(ip_)`, that is used to indicate whether the problem should be treated as dense and solved using dense-factorization methods. Possible values are:

- `0` the problem should be considered as sparse.
- `1` the problem should be considered as dense.
- `other` the algorithm will decide whether to treat the problem as dense or sparse depending on its dimension and the sparsity of the matrices involved.

The default is `dense_factorization = 0`.

`new_h` is a scalar variable of type `INTEGER(ip_)`, that is used to indicate how **H** has changed (if at all) since the previous call to `RQS_solve`. Possible values are:

- `0` **H** is unchanged.
- `1` the values in **H** have changed, but its nonzero structure is as before.
- `2` both the values and structure of **H** have changed.

The default is `new_h = 2`.

`new_m` is a scalar variable of type `INTEGER(ip_)`, that is used to indicate how **M** (if required) has changed (if at all) since the previous call to `RQS_solve`. Possible values are:

- `0` **M** is unchanged.
- `1` the values in **M** have changed, but its nonzero structure is as before.
- `2` both the values and structure of **M** have changed.

The default is `new_m = 2`.

`new_a` is a scalar variable of type `INTEGER(ip_)`, that is used to indicate how **A** (if required) has changed (if at all) since the previous call to `RQS_solve`. Possible values are:

- `0` **A** is unchanged.
- `1` the values in **A** have changed, but its nonzero structure is as before.
- `2` both the values and structure of **A** have changed.

The default is `new_a = 2`.

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

max_factorizations is a scalar variable of type INTEGER(ip_), that holds the maximum number of factorizations which will be permitted. If max_factorizations is set to a negative number, there will be no limit on the number of factorizations allowed. The default is max_factorizations = -1.

inverse_itmax is a scalar variable of type INTEGER(ip_), that holds the maximum number of inverse iterations which will be allowed per step when estimating the leftmost eigenvalue in RQS_solve. If inverse_itmax is set to a non-positive number, it will be reset by RQS_solve to 2. The default is inverse_itmax = 2.

taylor_max_degree is a scalar variable of type INTEGER(ip_), that specifies the maximum degree of Taylor approximant that will be used to approximate the secular function when trying to improve λ; a first-degree approximant results in Newton's method. The higher the degree, the better in general the improvement, but the larger the cost. Thus there is a balance between many cheap low-degree approximants and a few more expensive higher-degree ones. Our experience favours higher-degree approximants. The default is taylor_max_degree = 3, which is the highest degree currently supported.

initial_multiplier is a scalar variables of type REAL(rp_), that should be set to an initial estimate of the required multiplier λ_* (see Section 4). The algorithm will only use this value if %use_initial_multiplier is set .TRUE. (see below), and otherwise will be reset by RQS_solve. A good initial estimate may sometimes dramatically improve the performance of the package. The default is initial_multiplier = 0.0.

lower is a scalar variables of type REAL(rp_), that holds the value of any known lower bound on the required multiplier λ_*. A good lower bound may sometimes dramatically improve the performance of the package, but an incorrect value might cause the method to fail. Thus resetting lower from its default should be used with caution. The default is lower = - HUGE(1.0) (-HUGE(1.0D0) in GALAHAD_RQS_double).

upper is a scalar variables of type REAL(rp_), that holds the value of any known upper bound on the required multiplier λ_*. A good upper bound may sometimes dramatically improve the performance of the package, but an incorrect value might cause the method to fail. Thus resetting upper from its default should be used with caution. The default is upper = HUGE(1.0) (HUGE(1.0D0) in GALAHAD_RQS_double).

stop_normal and stop_hard are scalar variables of type REAL(rp_), that hold values for the standard convergence tolerances of the method (see Section 4). In particular, the method is deemed to have converged when the computed solution $\mathbf{x}$ and its multiplier λ satisfy $\|\mathbf{x}\|_{\mathbf{M}} - (\lambda/\sigma)^{1/(p-2)} \leq$ stop_normal $* \max(1, \|\mathbf{x}\|_{\mathbf{M}}, (\lambda/\sigma)^{1/(p-2)})$ or $\lambda_U - \lambda_L \leq$ stop_hard $* \max(1, |\lambda_L|, |\lambda_U|)$, where λ_L and λ_U are computed lower and upper bounds on the optimal multiplier λ_*. The defaults are stop_normal = stop_hard $= u^{0.75}$, where u is EPSILON(1.0) (EPSILON(1.0D0) in GALAHAD_RQS_double).

start_invit_tol is a scalar variable of type REAL(rp_), that holds the value of the starting tolerance for inverse iteration. Specifically, inverse iteration is started as soon as $\lambda_U - \lambda_L \leq$ start_invit_tol $* \max(|\lambda_L|, |\lambda_U|)$, where λ_L and λ_U are computed lower and upper bounds on the optimal multiplier λ_*. The default is start_invit_tol = 0.5.

start_invitmax_tol is a scalar variables of type REAL(rp_), that holds the value of the starting tolerance for full inverse iteration. Specifically, Specifically, inverse_itmax steps of inverse iteration are started as soon as $\lambda_U - \lambda_L \leq$ start_invitmax_tol $* \max(|\lambda_L|, |\lambda_U|)$, where λ_L and λ_U are computed lower and upper bounds on the optimal multiplier λ_*. The default is start_invitmax_tol = 0.1.

use_initial_multiplier is a scalar variable of type default LOGICAL, that may be set .TRUE. if the user wishes to use the value of initial multiplier supplied in %initial_multiplier, and .FALSE. if the initial value will be chosen automatically. The default is use_initial_multiplier = .FALSE..

initialize_approx_eigenvector is a scalar variable of type default LOGICAL, that be should set .TRUE. if the user wishes the package to choose an initial estimate of the eigenvector corresponding to the leftmost eigenvalue of the matrix pencil $(\mathbf{H}, \mathbf{M})$ in the null-space of $\mathbf{A}$. If the eigenvector corresponding to the previous

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

problem (if any) might be useful, initial_approx_eigenvector should be set .FALSE.. The default is initialize_approx_eigenvector = .TRUE..

space_critical is a scalar variable of type default LOGICAL, that may be set .TRUE. if the user wishes the package to allocate as little internal storage as possible, and .FALSE. otherwise. The package may be more efficient if space_critical is set .FALSE.. The default is space_critical = .FALSE..

deallocate_error_fatal is a scalar variable of type default LOGICAL, that may be set .TRUE. if the user wishes the package to return to the user in the unlikely event that an internal array deallocation fails, and .FALSE. if the package should be allowed to try to continue. The default is deallocate_error_fatal = .FALSE..

symmetric_linear_solver is a scalar variable of type default CHARACTER and length 30, that specifies the external package to be used to solve any symmetric linear system that might arise. Current possible choices are 'sils', 'ma27', 'ma57', 'ma77', 'ma86', 'ma97', ssids, 'pardiso' and 'wsmp', although only 'sils' and, for OMP 4.0-compliant compilers, 'ssids' are installed by default. See the documentation for the GALAHAD package SLS for further details. The default is symmetric_linear_solver = 'sils'.

definite_linear_solver is a scalar variable of type default CHARACTER and length 30, that specifies the external package to be used to solve any symmetric positive-definite linear system that might arise. Current possible choices are 'sils', 'ma27', 'ma57', 'ma77', 'ma86', 'ma87', 'ma97', ssids, 'pardiso' and 'wsmp', although only 'sils' and, for OMP 4.0-compliant compilers, 'ssids' are installed by default. See the documentation for the GALAHAD package SLS for further details. The default is definite_linear_solver = 'sils'.

prefix is a scalar variable of type default CHARACTER and length 30, that may be used to provide a user-selected character string to preface every line of printed output. Specifically, each line of output will be prefaced by the string prefix(2:LEN(TRIM(prefix))-1), thus ignoring the first and last non-null components of the supplied string. If the user does not want to preface lines by such a string, they may use the default prefix = "".

SLS_control is a scalar variable of type SLS_control_type that is used to control various aspects of the factorization package SLS. See the documentation for GALAHAD_SLS for more details.

IR_control is a scalar variable of type IR_control_type that is used to control various aspects of the iterative refinement package IR. See the documentation for GALAHAD_IR for more details.

2.4.3 The derived data type for holding history information

The derived data type RQS_history_type is used to hold the value of $\|\mathbf{x}(\lambda)\|_{\mathbf{M}}$, where $\mathbf{x}(\lambda)$ satisfies $(\mathbf{H}+\lambda\mathbf{M})\mathbf{x}(\lambda) = -\mathbf{c}$ and $\mathbf{A}\mathbf{x}(\lambda) = \mathbf{0}$ for a specific λ arising during the computation. The components of RQS_history_type are:

lambda is a scalar variable of type REAL(rp_), that gives the value λ.

x_norm is a scalar variable of type default REAL, that gives the corresponding value $\|\mathbf{x}(\lambda)\|_{\mathbf{M}}$.

2.4.4 The derived data type for holding timing information

The derived data type RQS_time_type is used to hold elapsed CPU and system clock times for the various parts of the calculation. The components of RQS_time_type are:

total is a scalar variable of type REAL(rp_), that gives the total CPU time spent in the package.

assemble is a scalar variable of type REAL(rp_), that gives the CPU time spent assembling the matrix (1.1) from its constituent parts.

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ for full details.

`analyse` is a scalar variable of type `REAL(rp_)`, that gives the CPU time spent analysing required matrices prior to factorization.

`factorize` is a scalar variable of type `REAL(rp_)`, that gives the CPU time spent factorizing the required matrices.

`solve` is a scalar variable of type `REAL(rp_)`, that gives the CPU time spent using the factors to solve relevant linear equations.

`clock_total` is a scalar variable of type `REAL(rp_)`, that gives the total elapsed system clock time spent in the package.

`clock_assemble` is a scalar variable of type `REAL(rp_)`, that gives the elapsed system clock time spent assembling the matrix (1.1) from its constituent parts.

`clock_analyse` is a scalar variable of type `REAL(rp_)`, that gives the elapsed system clock time spent analysing required matrices prior to factorization.

`clock_factorize` is a scalar variable of type `REAL(rp_)`, that gives the elapsed system clock time spent factorizing the required matrices.

`clock_solve` is a scalar variable of type `REAL(rp_)`, that gives the elapsed system clock time spent using the factors to solve relevant linear equations.

2.4.5 The derived data type for holding informational parameters

The derived data type `RQS_inform_type` is used to hold parameters that give information about the progress and needs of the algorithm. The components of `RQS_inform_type` are:

`status` is a scalar variable of type `INTEGER(ip_)`, that gives the current status of the algorithm. See Section 2.6 for details.

`alloc_status` is a scalar variable of type `INTEGER(ip_)`, that gives the status of the last internal array allocation or deallocation. This will be 0 if `status = 0`.

`bad_alloc` is a scalar variable of type default `CHARACTER` and length 80, that gives the name of the last internal array for which there were allocation or deallocation errors. This will be the null string if `status = 0`.

`factorizations` is a scalar variable of type `INTEGER(ip_)`, that gives the number of factorizations of the matrix (1.1) for different λ, performed during the calculation.

`max_entries_factors` is a scalar variable of type `INTEGER(ip_)`, that gives the maximum number of entries in any of the matrix factorizations performed during the calculation.

`len_history` is a scalar variable of type `INTEGER(ip_)`, that gives the number of $(\lambda, \|\mathbf{x}(\lambda)\|_{\mathbf{M}})$ pairs encountered during the calculation.

`obj` is a scalar variable of type `REAL(rp_)`, that holds the value of the quadratic function $\frac{1}{2}\mathbf{x}^T\mathbf{H}\mathbf{x} + \mathbf{c}^T\mathbf{x} + f$.

`obj_regularized` is a scalar variable of type `REAL(rp_)`, that holds the value of the regularized objective function $\frac{1}{2}\mathbf{x}^T\mathbf{H}\mathbf{x} + \mathbf{c}^T\mathbf{x} + f + \frac{1}{p}\sigma\|\mathbf{x}\|_{\mathbf{M}}^p$.

`multiplier` is a scalar variable of type `REAL(rp_)`, that holds the value of the Lagrange multiplier λ associated with the regularisation.

`x_norm` is a scalar variable of type `REAL(rp_)`, that holds the value of $\|\mathbf{x}\|_{\mathbf{M}}$.

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` for full details.

`pole` is a scalar variable of type `REAL(rp_)`, that holds a lower bound on $\max(0, -\lambda_1)$, where λ_1 is the left-most eigenvalue of the matrix pencil (H, M).

`hard_case` is a scalar variable of type default `LOGICAL`, that will be `.TRUE.` if the "hard-case" has occurred (see Section 4) and `.FALSE.` otherwise.

`time` is a scalar variable of type `RQS_time_type` whose components are used to hold elapsed CPU and system clock times for the various parts of the calculation (see Section 2.4.4).

`history` is an array argument of dimension `len_history` and type `RQS_history_type` that contains a list of pairs $(\lambda, \|\mathbf{x}(\lambda)\|_{\mathbf{M}})$ encountered during the calculation (see Section 2.4.3).

`SLS_inform` is a scalar variable of type `SLS_inform_type`, that holds informational parameters concerning the analysis, factorization and solution phases performed by the GALAHAD sparse matrix factorization package `SLS`. See the documentation for the package `SLS` for details of the derived type `SLS_inform_type`.

`IR_inform` is a scalar variable of type `IR_inform_type`, that holds informational parameters concerning the iterative refinement subroutine contained in the GALAHAD refinement package `IR`. See the documentation for the package `IR` for details of the derived type `IR_inform_type`.

2.4.6 The derived data type for holding problem data

The derived data type `RQS_data_type` is used to hold all the data for a particular problem between calls of `RQS` procedures. This data should be preserved, untouched, from the initial call to `RQS_initialize` to the final call to `RQS_terminate`.

2.5 Argument lists and calling sequences

There are three procedures for user calls (see Section 2.7 for further features):

1. The subroutine `RQS_initialize` is used to set default values and initialize private data.
2. The subroutine `RQS_solve` is called to solve the problem.
3. The subroutine `RQS_terminate` is provided to allow the user to automatically deallocate array components of the private data, allocated by `RQS_solve`, at the end of the solution process.

We use square brackets `[ ]` to indicate `OPTIONAL` arguments.

2.5.1 The initialization subroutine

Default values are provided as follows:

```
CALL RQS_initialize( data, control, inform )
```

`data` is a scalar `INTENT(INOUT)` argument of type `RQS_data_type` (see Section 2.4.6). It is used to hold data about the problem being solved.

`control` is a scalar `INTENT(OUT)` argument of type `RQS_control_type` (see Section 2.4.2). On exit, `control` contains default values for the components as described in Section 2.4.2. These values should only be changed after calling `RQS_initialize`.

`inform` is a scalar `INTENT(OUT)` argument of type `RQS_inform_type` (see Section 2.4.5). A successful call to `RQS_initialize` is indicated when the component `status` has the value 0. For other return values of `status`, see Section 2.6.

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

2.5.2 The optimization problem solution subroutine

The optimization problem solution algorithm is called as follows:

```
CALL RQS_solve( n, p, sigma, f, C, H, X, data, control, inform[, M, A] )
```

`n` is a scalar `INTENT(IN)` argument of type `INTEGER(ip_)`, that must be set to the number of unknowns, n. **Restriction:** $n > 0$.

`p` is a scalar `INTENT(IN)` variable of type `REAL(rp_)`, that must be set on initial entry to the order of the regularisation, p. **Restriction:** $p > 2$.

`sigma` is a scalar `INTENT(IN)` variable of type `REAL(rp_)`, that must be set on initial entry to the value of the regularisation weight, σ. **Restriction:** $\sigma > 0$.

`f` is a scalar `INTENT(IN)` variable of type `REAL(rp_)`, that holds the scalar value f for the objective function.

`C` is an array `INTENT(IN)` argument of dimension `n` and type `REAL(rp_)`, whose i-th entry holds the component c_i of the vector $\mathbf{c}$ for the objective function.

`H` is scalar `INTENT(IN)` argument of type `SMT_TYPE` that holds the Hessian matrix $\mathbf{H}$. The following components are used here:

`H%type` is an allocatable array of rank one and type default `CHARACTER`, that is used to indicate the storage scheme used. If the dense storage scheme (see Section 2.1.1) is used, the first five components of `H%type` must contain the string `DENSE`. For the sparse co-ordinate scheme (see Section 2.1.2), the first ten components of `H%type` must contain the string `COORDINATE`, for the sparse row-wise storage scheme (see Section 2.1.3), the first fourteen components of `H%type` must contain the string `SPARSE_BY_ROWS`, and for the diagonal storage scheme (see Section 2.1.4), the first eight components of `H%type` must contain the string `DIAGONAL`, for the scaled-identity matrix storage scheme (see Section 2.1.5), the first fifteen components of `H%type` must contain the string `SCALED_IDENTITY`, for the identity matrix storage scheme (see Section 2.1.6), the first eight components of `H%type` must contain the string `IDENTITY`, and for the zero matrix storage scheme (see Section 2.1.7), the first four components of `H%type` must contain the string `ZERO`.

For convenience, the procedure `SMT_put` may be used to allocate sufficient space and insert the required keyword into `H%type`. For example, if we wish to store $\mathbf{M}$ using the co-ordinate scheme, we may simply

```
CALL SMT_put( H%type, 'COORDINATE' )
```

See the documentation for the GALAHAD package `SMT` for further details on the use of `SMT_put`.

`H%ne` is a scalar variable of type `INTEGER(ip_)`, that holds the number of entries in the **lower triangular** part of $\mathbf{H}$ in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be set for any of the other three schemes.

`H%val` is a rank-one allocatable array of type `REAL(rp_)`, that holds the values of the entries of the **lower triangular** part of the Hessian matrix $\mathbf{H}$ in any of the storage schemes discussed in Section 2.1.

`H%row` is a rank-one allocatable array of type `INTEGER(ip_)`, that holds the row indices of the **lower triangular** part of $\mathbf{H}$ in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be allocated for any of the other three schemes.

`H%col` is a rank-one allocatable array variable of type `INTEGER(ip_)`, that holds the column indices of the **lower triangular** part of $\mathbf{H}$ in either the sparse co-ordinate (see Section 2.1.2), or the sparse row-wise (see Section 2.1.3) storage scheme. It need not be allocated when the dense or diagonal storage schemes are used.

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

H%ptr is a rank-one allocatable array of dimension n+1 and type INTEGER(ip_), that holds the starting position of each row of the **lower triangular** part of **H**, as well as the total number of entries plus one, in the sparse row-wise storage scheme (see Section 2.1.3). It need not be allocated when the other schemes are used.

X is an array INTENT(OUT) argument of dimension n and type REAL(rp_), that holds an estimate of the solution **x** of the problem on exit.

data is a scalar INTENT(INOUT) argument of type RQS_data_type (see Section 2.4.6). It is used to hold data about the problem being solved. It must not have been altered **by the user** since the last call to RQS_initialize.

control is a scalar INTENT(IN) argument of type RQS_control_type. (see Section 2.4.2). Default values may be assigned by calling RQS_initialize prior to the first call to RQS_solve.

inform is a scalar INTENT(INOUT) argument of type RQS_inform_type (see Section 2.4.5) whose components need not be set on entry. A successful call to RQS_solve is indicated when the component status has the value 0. For other return values of status, see Section 2.6.

M is an OPTIONAL scalar INTENT(IN) argument of type SMT_TYPE that holds the diagonally dominant scaling matrix **M**. It need only be set if $\mathbf{M} \neq \mathbf{I}$ and in this case the following components are used:

M%type is an allocatable array of rank one and type default CHARACTER, that is used to indicate the storage scheme used. If the dense storage scheme (see Section 2.1.1) is used, the first five components of M%type must contain the string DENSE. For the sparse co-ordinate scheme (see Section 2.1.2), the first ten components of M%type must contain the string COORDINATE, for the sparse row-wise storage scheme (see Section 2.1.3), the first fourteen components of M%type must contain the string SPARSE_BY_ROWS, and for the diagonal storage scheme (see Section 2.1.4), the first eight components of M%type must contain the string DIAGONAL, for the scaled-identity matrix storage scheme (see Section 2.1.5), the first fifteen components of M%type must contain the string SCALED_IDENTITY, for the identity matrix storage scheme (see Section 2.1.6), the first eight components of M%type must contain the string IDENTITY, and for the zero matrix storage scheme (see Section 2.1.7), the first four components of M%type must contain the string ZERO.

For convenience, the procedure SMT_put may be used to allocate sufficient space and insert the required keyword into M%type. For example, if we wish to store **M** using the co-ordinate scheme, we may simply

```
CALL SMT_put( M%type, 'COORDINATE' )
```

See the documentation for the GALAHAD package SMT for further details on the use of SMT_put.

M%ne is a scalar variable of type INTEGER(ip_), that holds the number of entries in the **lower triangular** part of **M** in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be set for any of the other three schemes.

M%val is a rank-one allocatable array of type REAL(rp_), that holds the values of the entries of the **lower triangular** part of the scaling matrix **M** in any of the storage schemes discussed in Section 2.1.

M%row is a rank-one allocatable array of type INTEGER(ip_), that holds the row indices of the **lower triangular** part of **M** in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be allocated for any of the other three schemes.

M%col is a rank-one allocatable array variable of type INTEGER(ip_), that holds the column indices of the **lower triangular** part of **M** in either the sparse co-ordinate (see Section 2.1.2), or the sparse row-wise (see Section 2.1.3) storage scheme. It need not be allocated when the dense or diagonal storage schemes are used.

M%ptr is a rank-one allocatable array of dimension n+1 and type INTEGER(ip_), that holds the starting position of each row of the **lower triangular** part of **M**, as well as the total number of entries plus one, in the sparse row-wise storage scheme (see Section 2.1.3). It need not be allocated when the other schemes are used.

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

If M is absent, the ℓ_2-norm, $\|\mathbf{x}\|_2 = \sqrt{\mathbf{x}^T\mathbf{x}}$, will be employed.

A is an OPTIONAL scalar INTENT(IN) argument of type SMT_TYPE that holds the constraint matrix **A**. It need only be set if the constraints $\mathbf{Ax} = \mathbf{0}$ are required, and in this case the following components are used:

A%type is an allocatable array of rank one and type default CHARACTER, that is used to indicate the storage scheme used. If the dense storage scheme (see Section 2.1.1) is used, the first five components of A%type must contain the string DENSE. For the sparse co-ordinate scheme (see Section 2.1.2), the first ten components of A%type must contain the string COORDINATE, for the sparse row-wise storage scheme (see Section 2.1.3), the first fourteen components of A%type must contain the string SPARSE_BY_ROWS, and for the zero matrix storage scheme (see Section 2.1.7), the first four components of A%type must contain the string ZERO.

For convenience, the procedure SMT_put may be used to allocate sufficient space and insert the required keyword into A%type. For example, if we wish to store **A** using the co-ordinate scheme, we may simply

```
CALL SMT_put( A%type, 'COORDINATE' )
```

See the documentation for the GALAHAD package SMT for further details on the use of SMT_put.

A%m is a scalar variable of type INTEGER(ip_), that holds the number of rows of **A**.

A%ne is a scalar variable of type INTEGER(ip_), that holds the number of entries in **A** in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be set for either of the other schemes.

A%val is a rank-one allocatable array of type REAL(rp_), that holds the values of the entries of the constraint matrix **A** in any of the storage schemes discussed in Section 2.1.

A%row is a rank-one allocatable array of type INTEGER(ip_), that holds the row indices of **A** in the sparse co-ordinate storage scheme (see Section 2.1.2). It need not be allocated for either of the other schemes.

A%col is a rank-one allocatable array variable of type INTEGER(ip_), that holds the column indices of **A** in either the sparse co-ordinate (see Section 2.1.2), or the sparse row-wise (see Section 2.1.3) storage scheme. It need not be allocated when the dense storage scheme is used.

A%ptr is a rank-one allocatable array of dimension A%m+1 and type INTEGER(ip_), that holds the starting position of each row of **A**, as well as the total number of entries plus one, in the sparse row-wise storage scheme (see Section 2.1.3). It need not be allocated when the other schemes are used.

2.5.3 The termination subroutine

All previously allocated arrays are deallocated as follows:

```
CALL RQS_terminate( data, control, inform )
```

data is a scalar INTENT(INOUT) argument of type RQS_data_type exactly as for RQS_solve that must not have been altered **by the user** since the last call to RQS_initialize. On exit, array components will have been deallocated.

control is a scalar INTENT(IN) argument of type RQS_control_type exactly as for RQS_solve.

inform is a scalar INTENT(OUT) argument of type RQS_inform_type exactly as for RQS_solve. Only the component status will be set on exit, and a successful call to RQS_terminate is indicated when this component status has the value 0. For other return values of status, see Section 2.6.

2.6 Warning and error messages

A negative value of inform%status on exit from RQS_solve or RQS_terminate indicates that an error might have occurred. No further calls should be made until the error has been corrected. Possible values are:

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

-1. An allocation error occurred. A message indicating the offending array is written on unit control%error, and the returned allocation status and a string containing the name of the offending array are held in inform%alloc_status and inform%bad_alloc, respectively.

-2. A deallocation error occurred. A message indicating the offending array is written on unit control%error and the returned allocation status and a string containing the name of the offending array are held in inform%alloc_status and inform%bad_alloc, respectively.

-3. (RQS_solve only) One of the restrictions n > 0, p > 2 or sigma > 0 has been violated.

-9. (RQS_solve only) The analysis phase of the factorization of the matrix (1.1) failed.

-10. (RQS_solve only) The factorization of the matrix (1.1) failed.

-15. (RQS_solve only) The matrix **M** appears not to be diagonally dominant.

-16. (RQS_solve only) The problem is so ill-conditioned that further progress is impossible.

-18. (RQS_solve only) Too many factorizations have been required. This may happen if control%max_factorizations is too small, but may also be symptomatic of a badly scaled problem.

2.7 Further features

In this section, we describe an alternative means of setting control parameters, that is components of the variable control of type RQS_control_type (see Section 2.4.2), by reading an appropriate data specification file using the subroutine RQS_read_specfile. This facility is useful as it allows a user to change RQS control parameters without editing and recompiling programs that call RQS.

A specification file, or specfile, is a data file containing a number of "specification commands". Each command occurs on a separate line, and comprises a "keyword", which is a string (in a close-to-natural language) used to identify a control parameter, and an (optional) "value", which defines the value to be assigned to the given control parameter. All keywords and values are case insensitive, keywords may be preceded by one or more blanks but values must not contain blanks, and each value must be separated from its keyword by at least one blank. Values must not contain more than 30 characters, and each line of the specfile is limited to 80 characters, including the blanks separating keyword and value.

The portion of the specification file used by RQS_read_specfile must start with a "BEGIN RQS" command and end with an "END" command. The syntax of the specfile is thus defined as follows:

```
( .. lines ignored by RQS_read_specfile .. )
  BEGIN RQS
     keyword    value
     .......    .....
     keyword    value
  END
( .. lines ignored by RQS_read_specfile .. )
```

where keyword and value are two strings separated by (at least) one blank. The "BEGIN RQS" and "END" delimiter command lines may contain additional (trailing) strings so long as such strings are separated by one or more blanks, so that lines such as

```
BEGIN RQS SPECIFICATION
```

and

```
END RQS SPECIFICATION
```

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

are acceptable. Furthermore, between the "BEGIN RQS" and "END" delimiters, specification commands may occur in any order. Blank lines and lines whose first non-blank character is ! or * are ignored. The content of a line after a ! or * character is also ignored (as is the ! or * character itself). This provides an easy manner to "comment out" some specification commands, or to comment specific values of certain control parameters.

The value of a control parameters may be of three different types, namely integer, logical or real. Integer and real values may be expressed in any relevant Fortran integer and floating-point formats (respectively). Permitted values for logical parameters are "ON", "TRUE", ".TRUE.", "T", "YES", "Y", or "OFF", "NO", "N", "FALSE", ".FALSE." and "F". Empty values are also allowed for logical control parameters, and are interpreted as "TRUE".

The specification file must be open for input when RQS_read_specfile is called, and the associated device number passed to the routine in device (see below). Note that the corresponding file is REWINDed, which makes it possible to combine the specifications for more than one program/routine. For the same reason, the file is not closed by RQS_read_specfile.

Control parameters corresponding to the components SLS_control and IR_control may be changed by including additional sections enclosed by "BEGIN SLS" and "END SLS", and "BEGIN IR" and "END IR", respectively. See the specification sheets for the packages GALAHAD_SLS and GALAHAD_IR for further details.

2.7.1 To read control parameters from a specification file

Control parameters may be read from a file as follows:

```
CALL RQS_read_specfile( control, device )
```

control is a scalar INTENT(INOUT) argument of type RQS_control_type (see Section 2.4.2). Default values should have already been set, perhaps by calling RQS_initialize. On exit, individual components of control may have been changed according to the commands found in the specfile. Specfile commands and the component (see Section 2.4.2) of control that each affects are given in Table 2.1.

device is a scalar INTENT(IN) argument of type INTEGER(ip_), that must be set to the unit number on which the specfile has been opened. If device is not open, control will not be altered and execution will continue, but an error message will be printed on unit control%error.

2.8 Information printed

If control%print_level is positive, information about the progress of the algorithm will be printed on unit control%out. If control%print_level = 1, a single line of output will be produced for each iteration of the process. In the first phase of the algorithm, this will include the current estimate of the multiplier and known brackets on its optimal value. In the second phase, the residual $\|\mathbf{x}\|_{\mathbf{M}} - \sigma\|\mathbf{x}|_M^{p-2}$, the current estimate of the multiplier and the size of the correction will be printed. If control%print_level ≥ 2, this output will be increased to provide significant detail of each iteration. This extra output includes times for various phases.

3 GENERAL INFORMATION

Use of common: None.

Workspace: Provided automatically by the module.

Other routines called directly: None.

Other modules used directly: RQS_solve calls the GALAHAD packages GALAHAD_CLOCK, GALAHAD_SYMBOLS, GALAHAD_SPACE, GALAHAD_RAND, GALAHAD_NORMS, GALAHAD_ROOTS, GALAHAD_SPECFILE, GALAHAD_SLS, GALAHAD_IR and GALAHAD_MOP.

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**

command	component of `control`	value type
`error-printout-device`	`%error`	integer
`printout-device`	`%out`	integer
`print-level`	`%print_level`	integer
`use-dense-factorization`	`%dense_factorization`	integer
`has-h-changed`	`%new_h`	integer
`has-m-changed`	`%new_m`	integer
`has-a-changed`	`%new_a`	integer
`factorization-limit`	`%max_factorizations`	integer
`inverse-iteration-limit`	`%inverse_itmax`	integer
`max-degree-taylor-approximant`	`%taylor_max_degree`	integer
`initial-multiplier`	`%initial_multiplier`	real
`lower-bound-on-multiplier`	`%lower`	real
`upper-bound-on-multiplier`	`%upper`	real
`stop-normal-case`	`%stop_normal`	real
`stop-hard-case`	`%stop_hard`	real
`start-inverse-iteration-tolerance`	`%start_invit_tol`	real
`start-max-inverse-iteration-tolerance`	`%start_invitmax_tol`	real
`use-initial-multiplier`	`%use_initial_multiplier`	logical
`initialize-approximate-eigenvector`	`%initialize_approx_eigenvector`	real
`space-critical`	`%space_critical`	logical
`deallocate-error-fatal`	`%deallocate_error_fatal`	logical
`definite-linear-equation-solver`	`%definite_linear_solver`	character
`output-line-prefix`	`%prefix`	character

Table 2.1: Specfile commands and associated components of `control`.

Input/output: Output is under control of the arguments `control%error`, `control%out` and `control%print_level`.

Restrictions: $n > 0$, $\sigma > 0$, $p > 2$.

Portability: ISO Fortran 95 + TR 15581 or Fortran 2003. The package is thread-safe.

4 METHOD

The required solution $\mathbf{x}_*$ necessarily satisfies the optimality condition $\mathbf{H}\mathbf{x}_* + \lambda_*\mathbf{M}\mathbf{x}_* + \mathbf{A}^T\mathbf{y}_* + \mathbf{c} = \mathbf{0}$ and $\mathbf{A}\mathbf{x}_* = 0$, where $\lambda_* = \sigma\|\mathbf{x}_*\|^{p-2}$ is a Lagrange multiplier corresponding to the regularisation and $\mathbf{y}_*$ are Lagrange multipliers for the linear constraints $\mathbf{A}\mathbf{x} = 0$, if any. In addition in all cases, the matrix $\mathbf{H} + \lambda_*\mathbf{M}$ will be positive semi-definite on the null=-space of $\mathbf{A}$; in most instances it will actually be positive definite, but in special "hard" cases singularity is a possibility.

The method is iterative, and proceeds in two phases. Firstly, lower and upper bounds, λ_L and λ_U, on λ_* are computed using Gershgorin's theorems and other eigenvalue bounds. The first phase of the computation proceeds by progressively shrinking the bound interval $[\lambda_L, \lambda_U]$ until a value λ for which $\|\mathbf{x}(\lambda)\|_{\mathbf{M}} \geq \sigma\|\mathbf{x}(\lambda)\|_M^{p-2}$ is found. Here $\mathbf{x}(\lambda)$ and its companion $\mathbf{y}(\lambda)$ are defined to be a solution of

$$(\mathbf{H} + \lambda\mathbf{M})\mathbf{x}(\lambda) + \mathbf{A}^T\mathbf{y}(\lambda) = -\mathbf{c} \text{ and } \mathbf{A}\mathbf{x}(\lambda) = \mathbf{0}. \tag{4.1}$$

Once the terminating λ from the first phase has been discovered, the second phase consists of applying Newton or higher-order iterations to the nonlinear "secular" equation $\lambda = \sigma\|\mathbf{x}(\lambda)\|_M^{p-2}$ with the knowledge that such iterations are both globally and ultimately rapidly convergent. It is possible in the "hard" case that the interval in the first-phase will

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

shrink to the single point λ_*, and precautions are taken, using inverse iteration with Rayleigh-quotient acceleration to ensure that this too happens rapidly.

The dominant cost is the requirement that we solve a sequence of linear systems (4.1). In the absence of linear constraints, an efficient sparse Cholesky factorization with precautions to detect indefinite $\mathbf{H}+\lambda\mathbf{M}$ is used. If $\mathbf{Ax}=0$ is required, a sparse symmetric, indefinite factorization of (1.1) is used rather than a Cholesky factorization.

Reference: The method is described in detail in

H. S. Dollar, N. I. M. Gould and D. P. Robinson. On solving trust-region and other regularised subproblems in optimization. *Mathematical Programming Computation* **2(1)** (2010) 21–57.

5 EXAMPLE OF USE

Suppose we wish to solve a problem in 10,000 unknowns, whose data is

$$\mathbf{H}=\begin{pmatrix} -2 & 1 & & & \\ 1 & -2 & . & & \\ & . & . & . & \\ & & . & -2 & 1 \\ & & & 1 & -2 \end{pmatrix},\ \mathbf{M}=\begin{pmatrix} 2 & & & & \\ & 2 & & & \\ & & . & & \\ & & & 2 & \\ & & & & 2 \end{pmatrix},\ \mathbf{c}=\begin{pmatrix} 1 \\ 1 \\ . \\ 1 \\ 1 \end{pmatrix} \text{ and } f=1,$$

with regularisation weight $\sigma=10$ and order $p=3$ but no other constraints. Then we may use the following code:

```
PROGRAM GALAHAD_RQS_EXAMPLE  !  GALAHAD 2.4 - 14/05/2010 AT 14:30 GMT.
USE GALAHAD_RQS_DOUBLE                       ! double precision version
IMPLICIT NONE
INTEGER, PARAMETER :: wp = KIND( 1.0D+0 )    ! set precision
INTEGER, PARAMETER :: n = 10000              ! problem dimension
REAL ( KIND = wp ), DIMENSION( n ) :: C, X
TYPE ( SMT_type ) :: H, M
TYPE ( RQS_data_type ) :: data
TYPE ( RQS_control_type ) :: control
TYPE ( RQS_inform_type ) :: inform
REAL ( KIND = wp ) :: f = 1.0_wp             ! constant term, f
REAL ( KIND = wp ) :: sigma = 10.0_wp        ! regularisation weight
REAL ( KIND = wp ) :: p = 3.0_wp             ! regularisation order
INTEGER :: i, s
C = 1.0_wp
CALL SMT_put( H%type, 'COORDINATE', s )      ! Specify co-ordinate for H
H%ne = 2 * n - 1
ALLOCATE( H%val( H%ne ), H%row( H%ne ), H%col( H%ne ) )
DO i = 1, n
 H%row( i ) = i ; H%col( i ) = i ; H%val( i ) = - 2.0_wp
END DO
DO i = 1, n - 1
 H%row( n + i ) = i + 1 ; H%col( n + i ) = i ; H%val( n + i ) = 1.0_wp
END DO
CALL SMT_put( M%type, 'DIAGONAL', s )         ! Specify diagonal for M
ALLOCATE( M%val( n ) ) ; M%val = 2.0_wp
CALL RQS_initialize( data, control, inform )  ! Initialize control parameters
CALL RQS_solve( n, p, sigma, f, C, H, X, data,                            &
                control, inform, M = M ) ! Solve
IF ( inform%status == 0 ) THEN !  Successful return
 WRITE( 6, "( 1X, I0,                                                     &
```

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

```
   ' factorizations. Objective and Lagrange multiplier =', 2ES12.4 )" )     &
       inform%factorizations, inform%obj, inform%multiplier
ELSE  !  Error returns
 WRITE( 6, "( ' RQS_solve exit status = ', I0 ) " ) inform%status
END IF
CALL RQS_terminate( data, control, inform )  ! delete internal workspace
DEALLOCATE( H%row, H%col, H%val, M%val )
END PROGRAM GALAHAD_RQS_EXAMPLE
```

This produces the following output:

```
 4 factorizations. Objective and Lagrange multiplier = -1.8703E+02  2.6592E+01
```

If we now add the constraint $\sum_{i=1}^{10000} x_i = 0$, for which $\mathbf{A} = (1 \ \ldots \ 1)$, but revert to unit ($\mathbf{M} = \mathbf{I}$) regularisation, we may solve the resulting problem using the following code:

```
PROGRAM GALAHAD_RQS_EXAMPLE2  !  GALAHAD 2.3 - 29/01/2009 AT 10:30 GMT.
USE GALAHAD_RQS_DOUBLE                      ! double precision version
IMPLICIT NONE
INTEGER, PARAMETER :: wp = KIND( 1.0D+0 )    ! set precision
INTEGER, PARAMETER :: n = 10000              ! problem dimension
REAL ( KIND = wp ), DIMENSION( n ) :: C, X
TYPE ( SMT_type ) :: H, A
TYPE ( RQS_data_type ) :: data
TYPE ( RQS_control_type ) :: control
TYPE ( RQS_inform_type ) :: inform
REAL ( KIND = wp ) :: f = 1.0_wp             ! constant term, f
REAL ( KIND = wp ) :: sigma = 10.0_wp        ! regularisation weight
REAL ( KIND = wp ) :: p = 3.0_wp             ! regularisation order
INTEGER :: i, s
C = 1.0_wp
CALL SMT_put( H%type, 'COORDINATE', s )      ! Specify co-ordinate for H
H%ne = 2 * n - 1
ALLOCATE( H%val( H%ne ), H%row( H%ne ), H%col( H%ne ) )
DO i = 1, n
 H%row( i ) = i ; H%col( i ) = i ; H%val( i ) = - 2.0_wp
END DO
DO i = 1, n - 1
 H%row( n + i ) = i + 1 ; H%col( n + i ) = i ; H%val( n + i ) = 1.0_wp
END DO
CALL SMT_put( A%type, 'DENSE', s )           ! Specify 1 by n matrix A
ALLOCATE( A%val( n ) ) ; A%val = 1.0_wp ; A%m = 1 ; A%n = n
DO i = 1, n
 A%val( i ) = REAL( i, KIND = wp )
END DO
CALL RQS_initialize( data, control, inform )  ! Initialize control parameters
CALL RQS_solve( n, p, sigma, f, C, H, X, data, control, inform, A = A )
IF ( inform%status == 0 ) THEN !  Successful return
 WRITE( 6, "( 1X, I0, ' factorizations. Objective and Lagrange multiplier',&
&  ' =',  2ES12.4 )" ) inform%factorizations, inform%obj, inform%multiplier
ELSE  !  Error returns
 WRITE( 6, "( ' RQS_solve exit status = ', I0 ) " ) inform%status
END IF
CALL RQS_terminate( data, control, inform )  ! delete internal workspace
DEALLOCATE( H%row, H%col, H%val, A%val )
END PROGRAM GALAHAD_RQS_EXAMPLE2
```

All use is subject to the conditions of a BSD-3-Clause License.
See `https://www.galahad.rl.ac.uk/download/` **for full details.**

This produces the following output:

```
5 factorizations. Objective and Lagrange multiplier = -1.1079E+02  2.2360E+01
```

All use is subject to the conditions of a BSD-3-Clause License.
See https://www.galahad.rl.ac.uk/download/ **for full details.**